

Your Agent Escaped Without Escaping the Sandbox

Unit tests are dead.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–21 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-00:50

Tell Tuesday as a scene. Pause before the last two sentences.

Tuesday. You give your agent a ticket: lock down admin. Anyone who isn't logged in gets a 401.

Assume your agent is Opus 5.5, the best coding model you can buy. It works in a sandbox with no network, and the test checker is locked.

A few minutes later, every test is green. And admin still lets anyone in.

Unit tests are dead. Not because agents stopped writing them. Because agents write them.

2. All green. Door open.

00:50-01:35

Let the room compare PASS and 200. Say the scripted-agent line plainly, once.

We built that Tuesday in a lab and recorded it. Here's the result. The checker says PASS. Admin with no login returns 200. It should be 401.

One thing up front. In our lab the agent is a script. We played the agent, the way a pentester plays the attacker, so every step can be replayed. We didn't need a model to misbehave. We needed to know whether the pipeline would notice.

3. A test is a second opinion

01:35-02:45

Left panel, then right panel. Land the self-report line slowly.

Why did tests ever work? Because two people were involved. One person decided what correct means and wrote it down as a test. Someone else wrote the code. When they disagreed, the test went red, and that disagreement was the whole value.

With an agent, one author holds both pens. It writes the code, it writes the tests, and it has one goal: make it green. Even a perfectly honest agent grading its own work is giving you a self-report.

That's what I mean by dead. The test still runs. It just stopped being a second opinion.

4. When the answer is in reach

02:45-03:55

One breath per year. None of them broke out of anything.

This isn't hypothetical. It has a history.

In 2018, researchers reported that a program-repair system called GenProg learned to delete the file holding the expected outputs. With nothing to compare against, every test passed.

In 2024, Sakana's AI Scientist hit a time limit on its experiments. Instead of making its code faster, it tried to change its own time limit.

In 2025, on the SWE-bench benchmark, agents including a Claude Sonnet model ran git log and found the future commit that contained the fix.

None of them broke out of anything. The answer was within reach, and they reached for it.

5. We built the pipeline that should survive it

03:55-04:45

Point to the writable directory, then to DENIED.

So we built the pipeline that should survive that. A Linux sandbox with no network. The program and its expected answers sit in the agent's working directory, because that's where the work happens. The checker is on a read-only mount.

First the agent tried the obvious move: overwrite the checker. Denied. Good. Then the checker ran and said FAIL. Correct. The program was broken.

6. Three suspects

04:45-06:00

One suspect at a time. Point at 401, then 200. Pause before the last line.

Then the check went green, on the same broken program. So who did it?

Suspect one, the program. We fingerprinted it before and after. Same bytes. Still returns 200.

Suspect two, the checker. Same bytes, and the overwrite had been denied.

Suspect three, the expected answers, the file the checker compares against. It changed. For admin without a login, 401 became 200. In fact every expected answer became 200.

So the agent didn't make the code pass the test. It made the test pass the code.

7. The flaw, in four lines

06:00-07:10

Count the four lines. Stress line four.

Here's the flaw in four lines. The checker was locked. But it read its expected answers from the workspace. The workspace belongs to the agent. And writing tests is the agent's job, so you can't take that permission away.

That last line is why this isn't just a misconfiguration. Least privilege can remove a token or a mount. It can't remove the agent's pen, because writing tests is the work. What you can change is whose copy counts.

And look at the record. Every file was fingerprinted. The change is right there in our own log. Nothing asked who was allowed to make it.

8. Separate the pens

07:10-08:20

Read the three rows top to bottom. Pause on REJECT.

The fix is to separate the pens. The agent writes the code, and it can propose tests. Something it can't write decides which tests count.

In our lab, the controller keeps its own copy of the expected answers. The same broken program: rejected. The corrected program: accepted.

And no, replacing the test with an AI reviewer doesn't fix this. The agent writes the code, the comments and the description that the reviewer reads. That's an argument, not something we measured. But it's the same shape: the thing being judged writes what the judge reads.

9. Even our own test lied

08:20-09:30

Slow down for the first test. Then read the four rows in order.

Now the humbling part. Our release gate takes one approval, for one artifact and one destination, used once. We wrote a test to prove it refuses substituted code.

The test passed. But it tried the wrong code after the approval had already been used up. The refusal proved nothing about the code check.

So we changed the order. Wrong code first, while the approval is unused: denied. Wrong destination: denied. Right code, right place: published. Then reuse: denied.

A no only counts as evidence when a matching yes works.

10. Every yes outside the sandbox is a test

09:30-10:45

One breath per row: the yes, our lab, the real case.

Tests aren't the only thing outside the sandbox that says yes. Every system that accepts what the agent hands it is a kind of test.

A service accepts a token. In our lab, a broad token read another project. In 2025, a compromised GitHub Action called tj-actions dumped the secrets it inherited into public build logs.

The next job trusts a shared file. Ours let one job change another's input.

An upload service accepts a request. Ours stored data in another account. Last year, CamoLeak moved private code out through GitHub's own allowed image proxy.

We recorded all four both ways: the unearned yes, then the fix, with the real job still working.

11. The benchmark builders just learned this

10:45-11:30

Point at the two numbers. Read the quote exactly.

Last month, the people who build coding benchmarks hit the same wall. A team auditing SWE-Bench Pro found that agents could read hidden tests, future commits and upstream fixes. One model's score fell from 79 percent to 57 once those were out of reach.

Their fix? Remove the hidden evaluation files from the agent's workspace. Separate the pens.

12. It escaped the test

11:30-12:15

Callback to Tuesday. Say the title line, pause, then the question. Stop.

Back to Tuesday. The sandbox held. The checker held. The agent never broke out of anything.

Your agent didn't escape the sandbox. It escaped the test.

So on Monday, pick one green check your team trusts, and ask one question: could the agent have written what made it green?

Everything we showed is on GitHub, with the recordings. Thank you.

Questions

13. What this experiment establishes

Questions

Use only in questions.

We measured whether these actions were allowed and whether the changed configuration stopped them. We didn't measure how often a model would try them, and we did not run Opus 5.5; it is the assumption in the story. The five cases on the right are the whole policy. Passing them shows behavior on those five inputs and nothing broader.

14. The token reached another project

Questions

Use only in questions.

The service enforced a token restricted to the task. The intended input stayed readable. A shorter lifetime alone does not narrow permissions: a broad token can reach everything it covers for as long as it is valid. Scope and audience have to be enforced by the service that receives the token.

15. The shared file changed on the host

Questions

Use only in questions.

The change persisted in the next-job fixture on the host. We did not run a later job that consumed it, so that part is an implication, not a recording. The same write through a read-only bind was denied, and the worker's own output directory stayed writable.

16. The upload landed in another account

Questions

Use only in questions.

Reaching a service does not establish permission for every operation or recipient on it. The receiver recorded the stored payload, so this is delivery, not an attempted send. In the corrected path the broker sends one fixed report to the team account and refuses extra recipient, payload or URL fields. This is a loopback service model, not a tested firewall bypass.

17. A bug in our earlier release gate

Questions

Use only in questions.

The earlier gate trusted a stored digest after the candidate bytes changed. Reproducing it required direct access to the controller's object. We did not show a worker-to-controller exploit. The current gate derives the digest from the bytes it publishes and stores those bytes.

18. What we recorded

Questions

Use only in questions.

These counts describe checks, not vulnerability frequency. Each record has its own scope. The 49 are regression tests for the controller and the evidence validator, not isolation tests. The old gate audit is one reproduction of our own bug.

19. Run the experiment

Questions

Use only in questions.

The Linux experiments need a disposable Linux machine with bubblewrap, Python 3.10 or later, and make. TeX Live is only needed to rebuild the PDFs, and GitHub Actions rebuilds them on every change. On any operating system, make test and make factory run the local protocol checks without OS isolation. No cloud account or token is needed.

20. What the approval covers

Questions

Use only in questions.

The controller checks the outputs, then authenticates its decision together with the candidate, criteria, and destination bindings, using a local HMAC key. The approval names the task and the run, expires, and can be used once. It does not sign the raw observations. The full field list is in factory core.

21. References

Questions

Use only in questions.

These are the sources for every real-world case in the talk. Each one is someone else's measurement or report; none of their numbers are transferred to our experiment. Full links are in the speaker notes and in the research dossier.